

Project Documentation: Song Playlist Generator

This document describes the implementation details and design decisions for the C program that generates a shuffled song playlist.

Input Reading Function

I used `fgets()` to read input from the command line (`stdin`). This function reads a line of text safely, including spaces, and prevents buffer overflows by limiting the read size. It's suitable for user input in a console application.

Example code:

```
12 void inputArtists(Artist artists[], int *artistCount) {
13     char buffer[MAX_SIZE];
14     int artistIndex = 0;
15
16     // Loop until max artists or empty input
17     while (artistIndex < MAX_ARTISTS) {
18         printf("Insert an artist/group name: ");
19         fgets(buffer, sizeof(buffer), stdin);
```

```
30         // Remove newline and copy name
31         buffer[strcspn(buffer, "\n")] = '\0';
```

Sorting Algorithm

I chose bubble sort for sorting songs within each artist alphabetically and then sorting artists by name. Bubble sort is simple to implement and works well for small datasets (up to 4 artists with 3 songs each). No need for more complex algorithms like quicksort for this scale.

```
7     for (int i = 0; i < artistCount; i++) {
8         for (int a = 0; a < artists[i].noOfSongs - 1; a++) {
9             for (int b = 0; b < artists[i].noOfSongs - 1 - a; b++) {
10                // Compare song titles
11                if (strcmp(artists[i].Songs[b], artists[i].Songs[b + 1]) > 0) {
12                    // Swap songs if out of order
13                    char tmp[MAX_SIZE];
14                    strcpy(tmp, artists[i].Songs[b]);
15                    strcpy(artists[i].Songs[b], artists[i].Songs[b + 1]);
16                    strcpy(artists[i].Songs[b + 1], tmp);
17                }
18            }
19        }
20    }
```

Shuffling Algorithm

Implementing the Fisher-Yates shuffle was not challenging. The real challenge came with implementing the specific modifications that we were given to handle. The playlist is duplicated first, then shuffled until it meets the constraint (same song not within 5 positions).

```
79         // Fisher-Yates shuffle
80         for (int i = newSize - 1; i > 0; i--) {
81             int j = rand() % (i + 1);
82             // Swap elements
83             Song temp = playlist[i];
84             playlist[i] = playlist[j];
85             playlist[j] = temp;
86         }
87         // Repeat if not valid
88         } while (!isValid(playlist, newSize));
89
```

The shuffle repeats in a loop until isValid() returns true, checking the distance between duplicate songs.